

Machine Learning in Flood Disaster Management

(Not Prediction) — Step-by-Step Project Build Guide

This guide walks through building the full flood disaster management ML system described in the original project outline, with one change: the YOLO victim-detection model is replaced with , a lighter, pretrained, and more readily available alternative. All other modules, models, and goals remain exactly as originally scoped.

Module	Purpose	Model Used
Flood Severity Assessment	Identify flooded areas & estimate severity	CNN
Rescue Priority Identification	Rank areas needing urgent rescue	Random Forest
Resource Allocation	Optimize food/water/medicine distribution	XGBoost
Area Severity Clustering	Group areas by severity level	K-Means
SOS Request Classification	Classify emergency text messages	NLP (DistilBERT)
Damage Assessment	Detect damaged buildings/roads/bridges	CNN
Safe Route Recommendation	Suggest safer rescue routes	Graph/road-network algorithms

Why Replace YOLO — and With What

YOLO delivers strong real-time detection accuracy, but it typically needs a labeled training dataset and GPU access to get good results quickly, which can be a barrier for a first build.

Step-by-Step Build Plan

Step 1 — Define Scope and Modules

Treat the 8 applications (severity assessment, rescue priority, SOS classification, damage assessment, route recommendation, resource allocation, shelter management) as separate mini-pipelines feeding one central dashboard. Decide which are must-haves for a first working version versus stretch goals for later.

Step 2 — Collect and Organize Datasets

Satellite/drone imagery: use public flood datasets such as Sen1Floods11 or FloodNet for CNN training. SOS text: use disaster-tweet datasets such as CrisisNLP, or simulate sample messages. Road/area data: OpenStreetMap for the safe-route module. Keep each dataset in its own folder mapped to its respective module.

Step 3 — Build the CNN for Flood Severity

Train or fine-tune a CNN (a ResNet-style classifier or a U-Net-style segmentation model) on satellite/drone images to output flooded-area masks and a severity score per region.

Step 4 — Build the Tabular / Classical ML Models

Random Forest for rescue-priority ranking, using features like population density, SOS count, vulnerability index, and accessibility. XGBoost for resource allocation optimization. K-Means for clustering areas by severity level. All three run well with scikit-learn — no deep learning required.

Step 5 — Build the NLP SOS Classifier

Fine-tune a lightweight transformer — DistilBERT trains faster than full BERT and is still readily available via Hugging Face — to classify incoming SOS text into medical emergency, evacuation, food/water need, or missing person categories.

Step 6 — Integrate Everything into One Pipeline

Write a central orchestration script (or a simple backend service) that accepts raw inputs (images, SOS text, location data), routes each to the correct model, and merges the outputs into one unified situation report per affected area.

Step 7 — Build a Dashboard for Authorities

A simple web dashboard (Flask plus a lightweight frontend) displaying flooded-area maps, ranked rescue priorities, detected victims, and resource-allocation suggestions. This is the piece that makes the project demoable end-to-end.

Suggested Starting Point

Rather than building all 7 modules at once, get one working end-to-end with sample data first —the Random Forest rescue-priority model are good starting points. Once that pipeline runs cleanly from input to output, add the remaining modules one at a time and wire each into the shared dashboard.